

innovate

achieve

lead



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

SEZG651/ SSZG653 Software Architectures

CS02 Quality Attributes 1 of 2

August 1, 2026

SEZG651/SSZG653 Software
Architectures

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Index



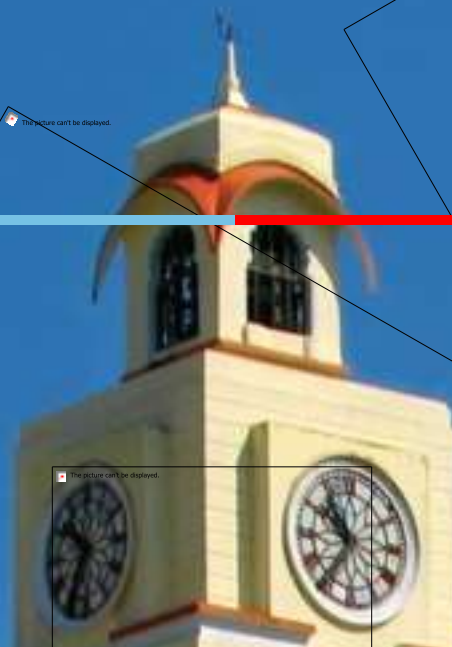
Quality Attribute Scenario

- Source of Stimulus
- Stimulus
- Artifact
- Environment
- Response
- Response Measure

Management Action

- Allocation of responsibilities
- Coordination model
- Data model
- Management of resources
- Mapping among architectural elements
- Binding time decisions
- Choice of technology

- Understanding Quality Attributes
- Availability
- Performance
- Usability
- Security
- Modifiability



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Understanding Quality Attributes

Module 2 – L1

Harvinder S Jabbal
CSIS, Work Integrated Learning Programs

August 1, 2026

SEZG651/SSZG653 Software
Architectures

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Understanding Quality Attributes



- Architecture and Requirements
- Functionality
- Quality Attribute Considerations
- Specifying Quality Attribute Requirements
- Achieving Quality Attributes through Tactics
- Guiding Quality Design Decisions

Architecture and Requirements



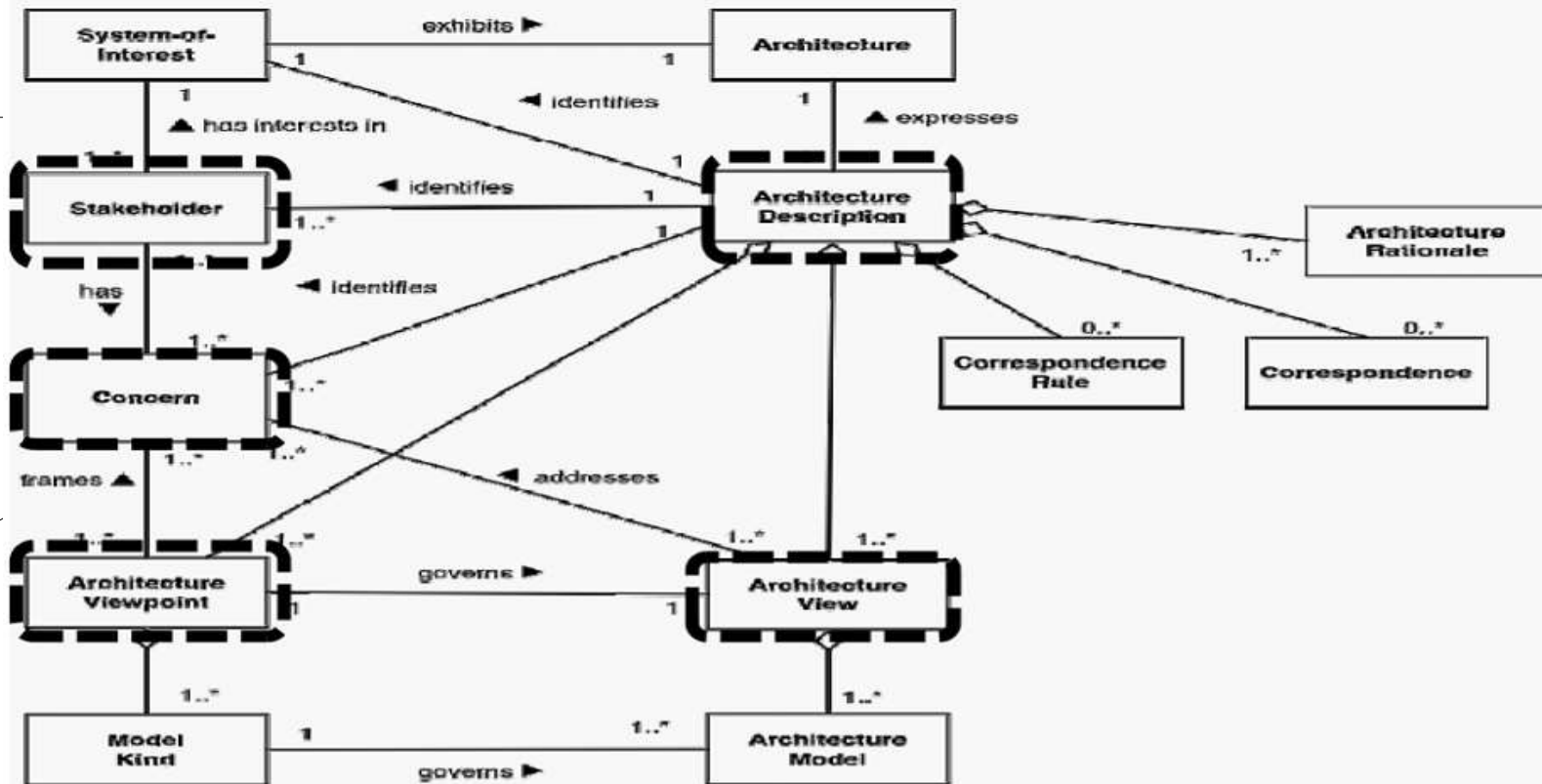
System requirements can be categorized as:

- Functional requirements. These requirements state what the system must do, how it must behave or react to run-time stimuli.
- Quality attribute requirements. These requirements annotate (qualify) functional requirements. Qualification might be how fast the function must be performed, how resilient it must be to erroneous input, how easy the function is to learn, etc.
- Constraints. A constraint is a design decision with zero degrees of freedom. That is, it's a design decision that has already been made for you.

Architectural Description



ISO/IEC/IEEE 42010: 2011



ISO/IEC/IEEE 42010 - International Standard for Systems and Software Engineering – Architectural Description, 2011

Functionality



- Functionality is the ability of the system to do the work for which it was intended.
- Functionality has a strange relationship to architecture:
 - functionality does not determine architecture; given a set of required functionality, there is no end to the architectures you could create to satisfy that functionality
 - functionality and quality attributes are orthogonal

Quality Attribute Considerations



- If a functional requirement is "when the user presses the green button the Options dialog appears":
 - a performance QA annotation might describe how quickly the dialog will appear;
 - an availability QA annotation might describe how often this function will fail, and how quickly it will be repaired;
 - a usability QA annotation might describe how easy it is to learn this function.

Quality Attribute Considerations



There are three problems with previous discussions of quality attributes:

1. The definitions provided for an attribute are not testable. It is meaningless to say that a system will be “modifiable”.
2. Endless time is wasted on arguing over which quality a concern belongs to. Is a system failure due to a denial of service attack an aspect of availability, performance, security, or usability?
3. Each attribute community has developed its own vocabulary.

Quality Attribute Considerations

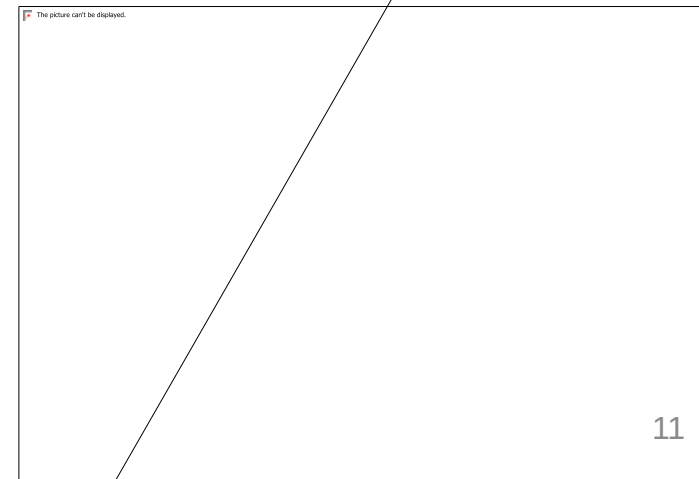
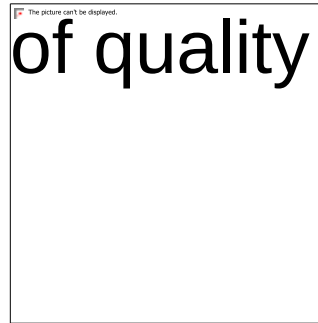


- A solution to the first two of these problems (untestable definitions and overlapping concerns) is to use *quality attribute scenarios* as a means of characterizing quality attributes.
- A solution to the third problem is to provide a discussion of each attribute—concentrating on its underlying concerns—to illustrate the concepts that are fundamental to that attribute community.

Specifying Quality Attribute Requirements



- We use a common form to specify all quality attribute requirements as scenarios.
- Our representation of quality attribute scenarios has these parts:
 1. **Stimulus**
 2. **Stimulus source**
 3. **Response**
 4. **Response measure**
 5. **Environment**
 6. **Artifact**



Specifying Quality Attribute Requirements



1. **Source of stimulus.** This is some entity (a human, a computer system, or any other actuator) that generated the stimulus.
2. **Stimulus.** The stimulus is a condition that requires a response when it arrives at a system.
3. **Environment.** The stimulus occurs under certain conditions. The system may be in an overload condition or in normal operation, or some other relevant state. For many systems, “normal” operation can refer to one of a number of modes.
4. **Artifact.** Some artifact is stimulated. This may be a collection of systems, the whole system, or some piece or pieces of it.
5. **Response.** The response is the activity undertaken as the result of the arrival of the stimulus.
6. **Response measure.** When the response occurs, it should be measurable in some fashion so that the requirement can be tested.

Specifying Quality Attribute Requirements

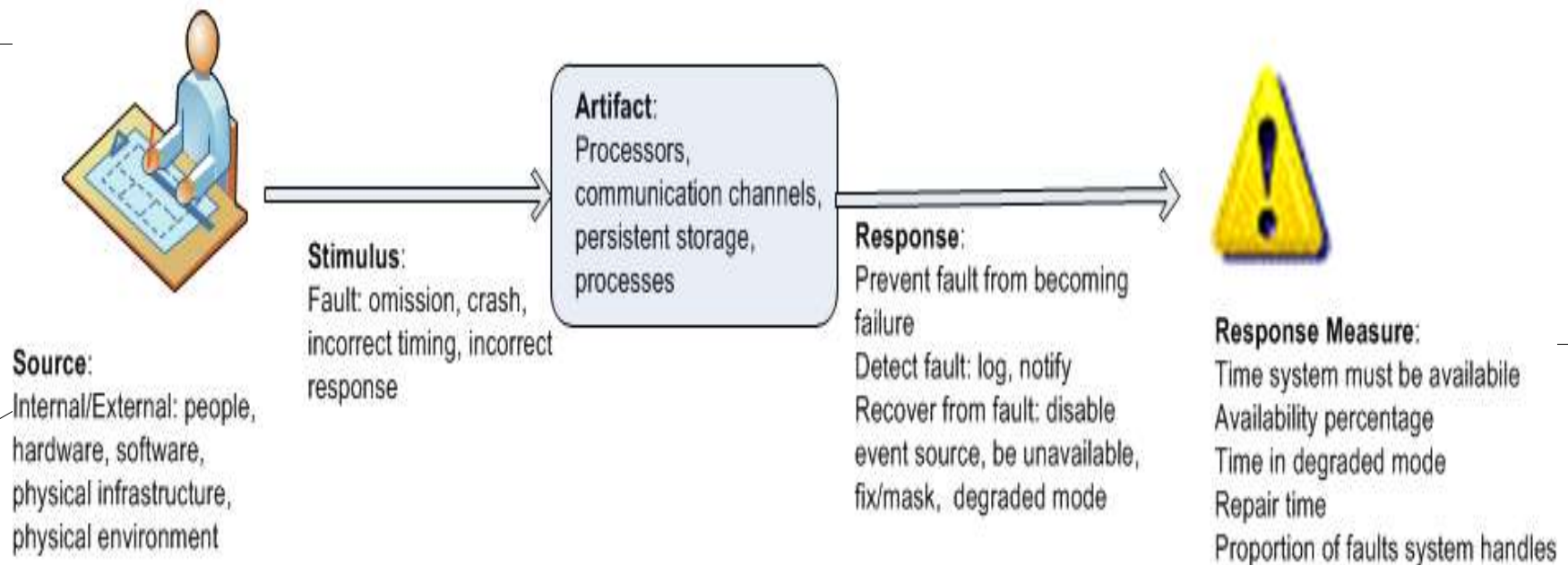


- We distinguish *general* quality attribute scenarios (“general scenarios”)—those that are system independent and can, potentially, pertain to any system—from *concrete* quality attribute scenarios (concrete scenarios)—those that are specific to the particular system under consideration.

Specifying Quality Attribute Requirements



Example general scenario for availability:



Achieving Quality Attributes Through Tactics



- There are a collection of primitive design techniques that an architect can use to achieve a quality attribute response.
- We call these architectural design primitive *tactics*.
- Tactics, like design patterns, are techniques that architects have been using for years. We do not *invent* tactics; we simply capture what architects do in practice.

Achieving Quality Attributes Through Tactics



Why do we do this? There are three reasons:

1. Design patterns are complex; they are a bundle of design decisions. But patterns are often difficult to apply as is; architects need to modify and adapt them. By understanding tactics, an architect can assess the options for augmenting an existing pattern to achieve a quality attribute goal.
2. If no pattern exists to realize the architect's design goal, tactics allow the architect to construct a design fragment from "first principles".
3. By cataloguing tactics, we make design more systematic. You frequently will have a choice of multiple tactics to improve a particular quality attribute. The choice of which tactic to use depends on factors such as tradeoffs among other quality attributes and the cost to implement.

Guiding Quality Design Decisions



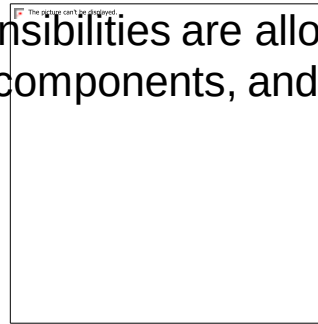
- Architecture design is a systematic approach to making design decisions.
- We categorize the design decisions that an architect needs to make as follows:
 1. Allocation of responsibilities
 2. Coordination model
 3. Data model
 4. Management of resources
 5. Mapping among architectural elements
 6. Binding time decisions
 7. Choice of technology

Allocation of Responsibilities



Decisions involving allocation of responsibilities include:

- identifying the important responsibilities including basic system functions, architectural infrastructure, and satisfaction of quality attributes.
- determining how these responsibilities are allocated to non-runtime and runtime elements (namely, modules, components, and connectors).



The picture can't be displayed.

Coordination Model



Decisions about the coordination model include:

- identify the elements of the system that must coordinate, or are prohibited from coordinating
- determining the properties of the coordination, such as timeliness, currency, completeness, correctness, and consistency
- choosing the communication mechanisms that realize those properties. Important properties of the communication mechanisms include stateful vs. stateless, synchronous vs. asynchronous, guaranteed vs. non-guaranteed delivery, and performance-related properties such as throughput and latency

Data Model



Decisions about the data model include:

- choosing the major data abstractions, their operations, and their properties. This includes determining how the data items are created, initialized, accessed, persisted, manipulated, translated, and destroyed.
- metadata needed for consistent interpretation of the data
- organization of the data. This includes determining whether the data is going to be kept in a relational data base, a collection of objects or both

Management of Resources



Decisions for management of resources include:

- identifying the resources that must be managed and determining the limits for each
- determining which system element(s) manage each resource
- determining how resources are shared and the arbitration strategies employed when there is contention
- determining the impact of saturation on different resources.

Mapping Among Architectural Elements



Useful mappings include:

- the mapping of modules and runtime elements to each other—that is, the runtime elements that are created from each module; the modules that contain the code for each runtime element
- the assignment of runtime elements to processors
- the assignment of items in the data model to data stores
- the mapping of modules and runtime elements to units of delivery

Binding Time



- The decisions in the other categories have an associated binding time decision. Examples of such binding time decisions include:

- For allocation of responsibilities, you can have build-time selection of modules via a parameterized build script.
- For choice of coordination model, you can design run-time negotiation of protocols.
- For resource management you can design a system to accept new peripheral devices plugged in at run-time.
- For choice of technology, you can build an app-store for a smart phone that automatically downloads the appropriate version of the app.

Choice of Technology



Choice of technology decisions involve:

- deciding which technologies are available to realize the decisions made in the other categories
- determining whether the tools to support this technology (IDEs, simulators, testing tools, etc.) are adequate
- determining the extent of internal familiarity and external support for the technology (such as courses, tutorials, examples, availability of contractors)
- determining the side effects of choosing a technology such as a required coordination model or constrained resource management opportunities
- determining whether a new technology is compatible with the existing technology stack

Summary



Requirements for a system come in three categories.

1. Functional. These requirements are satisfied by including an appropriate set of responsibilities within the design.
2. Quality attribute. These requirements are satisfied by the structures and behaviors of the architecture.
3. Constraints. A constraint is a design decision that's already been made.

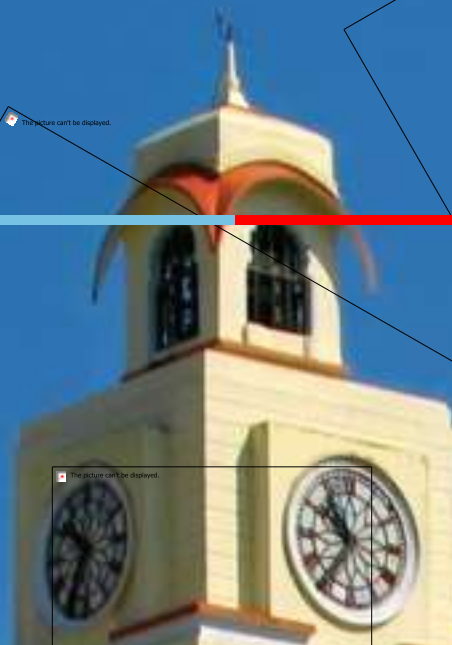
To express a quality attribute requirement we use a quality attribute scenario. The parts of the scenario are:

1. Source of stimulus.
2. Stimulus
3. Environment.
4. Artifact.
5. Response.
6. Response measure.

Summary



- An architectural tactic is a design decision that affects a quality attribute response. The focus of a tactic is on a single quality attribute response.
- Architectural patterns can be seen as “packages” of tactics.
- The seven categories of architectural design decisions are:
 1. Allocation of responsibilities
 2. Coordination model
 3. Data model
 4. Management of resources
 5. Mapping among architectural elements
 6. Binding time decisions
 7. Choice of technology



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Availability Module 2 – L2

Harvinder S Jabbal
CSIS, Work Integrated Learning Programs

August 1, 2026

SEZG651/SSZG653 Software
Architectures

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Outline



What is Availability?

Availability General Scenario

Tactics for Availability

A Design Checklist for Availability

Summary

Credits



These Slides are based on

- **Software Architecture in Practice** by
 - Len Bass, Paul Clement and Rick Kazman
 - Pearson © 2013

What is Availability?



- Availability refers to a property of software that it is there and ready to carry out its task when you need it to be.
- This is a broad perspective and encompasses what is normally called reliability.
- Availability builds on reliability by adding the notion of recovery (repair).
- Fundamentally, availability is about minimizing service outage time by mitigating faults.

Availability General Scenario



Portion of Scenario	Possible Values
Source	Internal/external: people, hardware, software, physical infrastructure, physical environment
Stimulus	Fault: omission, crash, incorrect timing, incorrect response
Artifact	System's processors, communication channels, persistent storage, processes
Environment	Normal operation, startup, shutdown, repair mode, degraded operation, overloaded operation
Response	Prevent the fault from becoming a failure Detect the fault: • log the fault • notify appropriate entities (people or systems) Recover from the fault • disable source of events causing the fault • be temporarily unavailable while repair is being effected • fix or mask the fault/failure or contain the damage it causes • operate in a degraded mode while repair is being effected
Response Measure	Time or time interval when the system must be available Availability percentage (e.g. 99.999%) Time to detect the fault Time to repair the fault Time or time interval in which system can be in degraded mode Proportion (e.g., 99%) or rate (e.g., up to 100 per second) of a certain class of faults that the system prevents, or handles without failing

Sample Concrete Availability Scenario



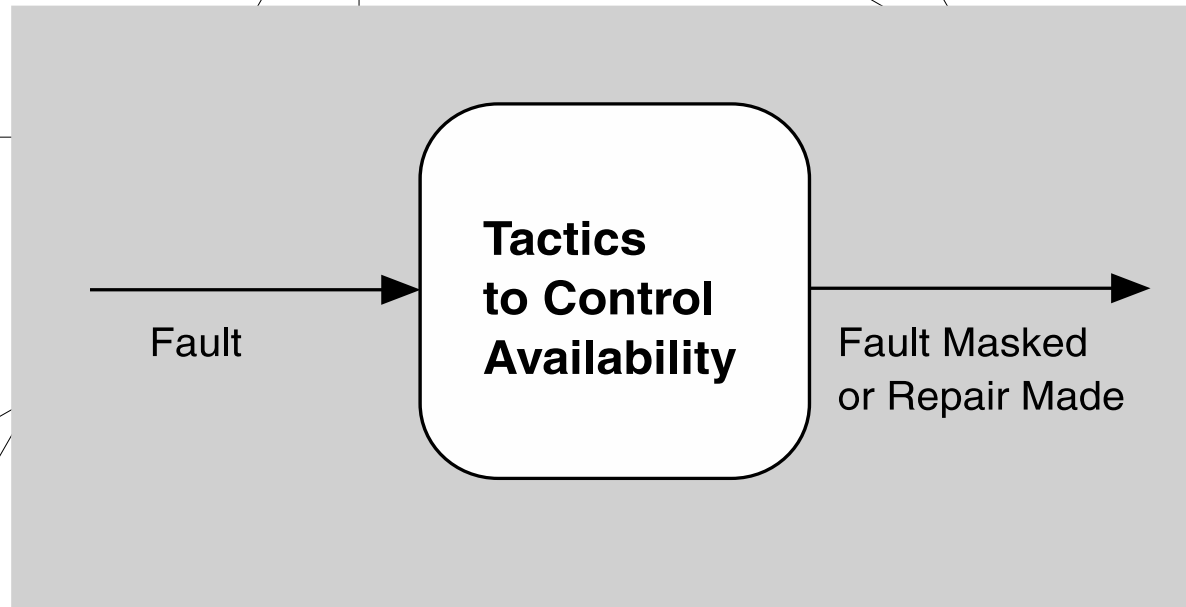
The heartbeat monitor determines that the server is nonresponsive during normal operations. The system informs the operator and continues to operate with no downtime.

Goal of Availability Tactics

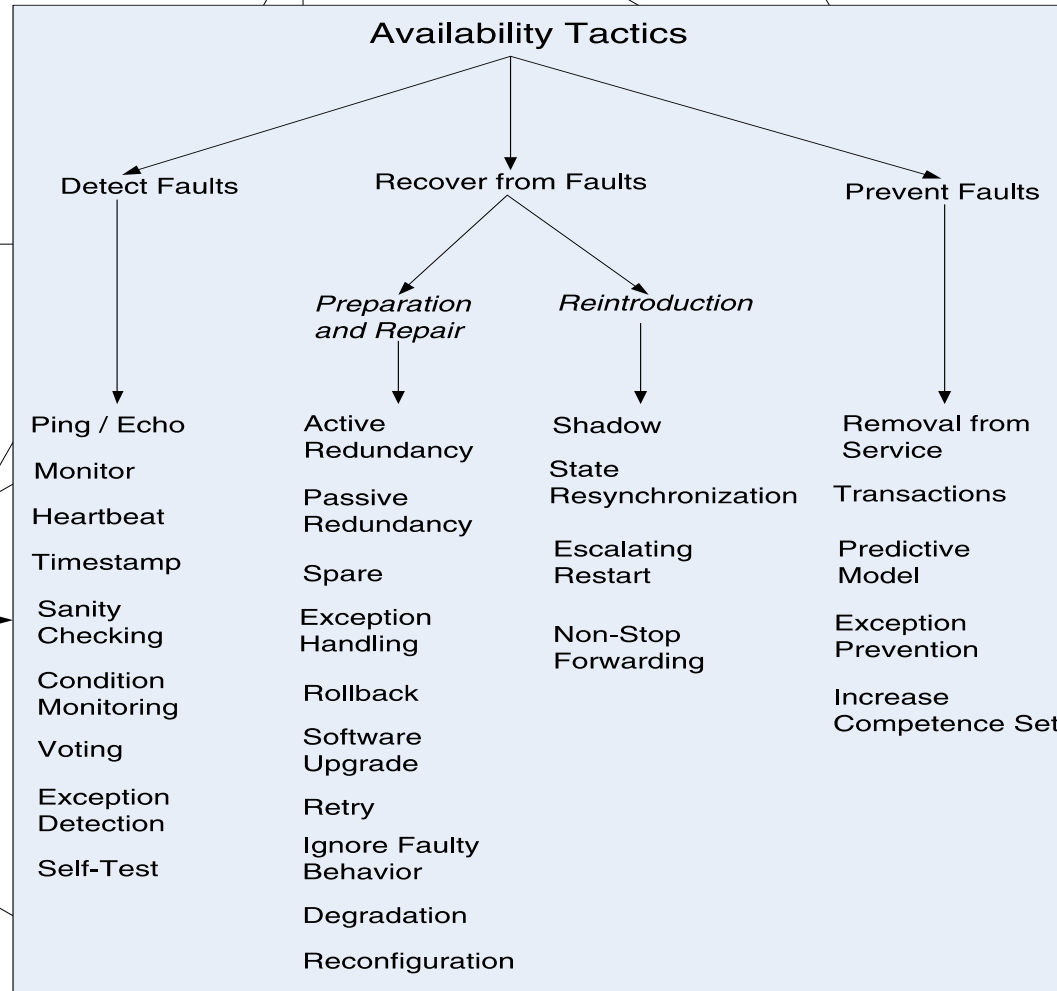


- A failure occurs when the system no longer delivers a service consistent with its specification
 - this failure is observable by the system's actors.
- A fault (or combination of faults) has the potential to cause a failure.
- Availability tactics enable a system to endure faults so that services remain compliant with their specifications.
- The tactics keep faults from becoming failures or at least bound the effects of the fault and make repair possible.

Goal of Availability Tactics



Availability Tactics



Detect Faults



- Ping/echo: asynchronous request/response message pair exchanged between nodes, used to determine reachability and the round-trip delay through the associated network path.
- Monitor: a component used to monitor the state of health of other parts of the system. A system monitor can detect failure or congestion in the network or other shared resources, such as from a denial-of-service attack.
- Heartbeat: a periodic message exchange between a system monitor and a process being monitored.

Detect Faults



- Timestamp: used to detect incorrect sequences of events, primarily in distributed message-passing systems.
- Sanity Checking: checks the validity or reasonableness of a component's operations or outputs; typically based on a knowledge of the internal design, the state of the system, or the nature of the information under scrutiny.
- Condition Monitoring: checking conditions in a process or device, or validating assumptions made during the design.

Detect Faults



- Voting: to check that replicated components are producing the same results. Comes in various flavors: replication, functional redundancy, analytic redundancy.
- Exception Detection: detection of a system condition that alters the normal flow of execution, e.g. system exception, parameter fence, parameter typing, timeout.
- Self-test: procedure for a component to test itself for correct operation.

Recover from Faults (Preparation & Repair)



- Active Redundancy (hot spare): all nodes in a *protection group* receive and process identical inputs in parallel, allowing redundant spare(s) to maintain synchronous state with the active node(s).
 - A protection group is a group of nodes where one or more nodes are “active,” with the remainder serving as redundant spares.
- Passive Redundancy (warm spare): only the active members of the protection group process input traffic; one of their duties is to provide the redundant spare(s) with periodic state updates.
- Spare (cold spare): redundant spares of a protection group remain out of service until a fail-over occurs, at which point a power-on-reset procedure is initiated on the redundant spare prior to its being placed in service.

Recover from Faults (Preparation & Repair)



- Exception Handling: dealing with the exception by reporting it or handling it, potentially masking the fault by correcting the cause of the exception and retrying.
- Rollback: revert to a previous known good state, referred to as the “rollback line”.
- Software Upgrade: in-service upgrades to executable code images in a non-service-affecting manner.

Recover from Faults (Preparation & Repair)



- **Retry:** where a failure is transient retrying the operation may lead to success.
- **Ignore Faulty Behavior:** ignoring messages sent from a source when it is determined that those messages are spurious.
- **Degradation:** maintains the most critical system functions in the presence of component failures, dropping less critical functions.
- **Reconfiguration:** reassigning responsibilities to the resources left functioning, while maintaining as much functionality as possible.

Recover from Faults (Reintroduction)



- Shadow: operating a previously failed or in-service upgraded component in a “shadow mode” for a predefined time prior to reverting the component back to an active role.
- State Resynchronization: partner to active redundancy and passive redundancy where state information is sent from active to standby components.
- Escalating Restart: recover from faults by varying the granularity of the component(s) restarted and minimizing the level of service affected.
- Non-stop Forwarding: functionality is split into supervisory and data. If a supervisor fails, a router continues forwarding packets along known routes while protocol information is recovered and validated.

Prevent Faults



- Removal From Service: temporarily placing a system component in an out-of-service state for the purpose of mitigating potential system failures
- Transactions: bundling state updates so that asynchronous messages exchanged between distributed components are *atomic, consistent, isolated, and durable*.
- Predictive Model: monitor the state of health of a process to ensure that the system is operating within nominal parameters; take corrective action when conditions are detected that are predictive of likely future faults.

Prevent Faults



- Exception Prevention: preventing system exceptions from occurring by masking a fault, or preventing it via smart pointers, abstract data types, wrappers.
- Increase Competence Set: designing a component to handle more cases—faults—as part of its normal operation.

Design Checklist for Availability



Allocation of Responsibilities

- Determine the system responsibilities that need to be highly available. Ensure that additional responsibilities have been allocated to detect an omission, crash, incorrect timing, or incorrect response.
- Ensure that there are responsibilities to:
 - log the fault
 - notify appropriate entities (people or systems)
 - disable source of events causing the fault
 - be temporarily unavailable
 - fix or mask the fault/failure
 - operate in a degraded mode

Design Checklist for Availability



Coordination Model

- Determine the system responsibilities that need to be highly available. With respect to those responsibilities
- Ensure that coordination mechanisms can detect an omission, crash, incorrect timing, or incorrect response. Consider, e.g., whether guaranteed delivery is necessary. Will the coordination work under degraded communication?
- Ensure that coordination mechanisms enable the logging of the fault, notification of appropriate entities, disabling of the source of the events causing the fault, fixing or masking the fault, or operating in a degraded mode
- Ensure that the coordination model supports the replacement of the artifacts (processors, communications channels, persistent storage, and processes). E.g., does replacement of a server allow the system to continue to operate?
- Determine if the coordination will work under conditions of degraded communication, at startup/shutdown, in repair mode, or under overloaded operation. E.g., how much lost information can the coordination model withstand and with what consequences?

Design Checklist for Availability



Data Model

- Determine which portions of the system need to be highly available. Within those portions, determine which data abstractions could cause a fault of omission, a crash, incorrect timing behavior, or an incorrect response.
- For those data abstractions, operations, and properties, ensure that they can be disabled, be temporarily unavailable, or be fixed or masked in the event of a fault.
- E.g., ensure that write requests are cached if a server is temporarily unavailable and performed when the server is returned to service.

Design Checklist for Availability



Mapping Among Architectural Elements

- Determine which artifacts (processors, communication channels, storage, processes) may produce a fault: omission, crash, incorrect timing, or incorrect response.
- Ensure that the mapping (or re-mapping) of architectural elements is flexible enough to permit the recovery from the fault. This may involve a consideration of
 - which processes on failed processors need to be re-assigned at runtime
 - which processors, data stores, or communication channels can be activated or re-assigned at runtime
 - how data on failed processors or storage can be served by replacement units
 - how quickly the system can be re-installed based on the units of delivery provided
 - how to (re-) assign runtime elements to processors, communication channels, and data stores
- When employing tactics that depend on redundancy of functionality, the mapping from modules to redundant components is important. E.g., it is possible to write a module that contains code appropriate for both the active and back-up components in a protection group.

Design Checklist for Availability



Resource Management

- Determine what critical resources are necessary to continue operating in the presence of a fault: omission, crash, incorrect timing, or incorrect response. Ensure there are sufficient remaining resources in the event of a fault to log the fault; notify appropriate entities (people or systems); disable source of events causing the fault; fix or mask the fault/failure; operate normally, in startup, shutdown, repair mode, degraded operation, and overloaded operation.
- Determine the availability time for critical resources, what critical resources must be available during specified time intervals, time intervals during which the critical resources may be in a degraded mode, and repair time for critical resources. Ensure that the critical resources are available during these time intervals.
- For example, ensure that input queues are large enough to buffer anticipated messages if a server fails so that the messages are not permanently lost.

Design Checklist for Availability



Binding Time

- Determine how and when architectural elements are bound. If late binding is used to alternate between components that can themselves be sources of faults (e.g. processes, processors, communication channels), ensure the chosen availability strategy is sufficient to cover faults introduced by all sources. E.g.
- If late binding is used to switch between processors that will be the subject of faults, will the fault detection and recovery mechanisms work for all possible bindings?
- If late binding is used to change the definition or tolerance of what constitutes a fault (e.g., how long a process can go without responding before a fault is assumed), is the recovery strategy chosen sufficient to handle all cases? For example, if a fault is flagged after 0.1 ms, but the recovery mechanism takes 1.5 seconds to work, that might be an unacceptable mismatch.
- What are the availability characteristics of the late binding mechanism itself? Can it fail?

Design Checklist for Availability



Choice of Technology

- Determine the available technologies that can (help) detect faults, recover from faults, re-introduce failed components.
- Determine what technologies are available that help the response to a fault (e.g., event loggers).
- Determine the availability characteristics of chosen technologies themselves: What faults can they recover from? What faults might they introduce into the system?

Summary

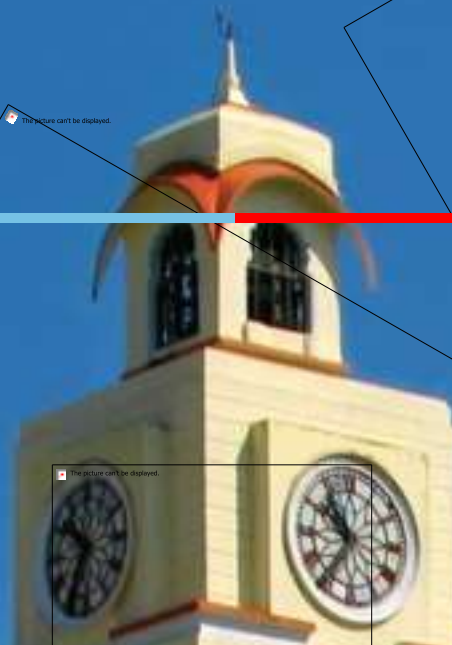


- Availability refers to the ability of the system to be available for use when a fault occurs.
- The fault must be recognized (or prevented) and then the system must respond.
- The response will depend on the criticality of the application and the type of fault
 - can range from “ignore it” to “keep on going as if it didn’t occur.”

Summary



- Tactics for availability are categorized into detect faults, recover from faults and prevent faults.
- Detection tactics depend on detecting signs of life from various components.
- Recovery tactics are retrying an operation or maintaining redundant data or computations.
- Prevention tactics depend on removing elements from service or limiting the scope of faults.
- All availability tactics involve the coordination model.



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Performance Module 2 – L3

Harvinder S Jabbal
CSIS, Work Integrated Learning Programs

August 1, 2026

SEZG651/SSZG653 Software
Architectures

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Chapter Outline



What is Performance?

Performance General Scenario

Tactics for Performance

A Design Checklist for Performance

Summary

What is Performance?



- Performance is about time and the software system's ability to meet timing requirements.
- When events occur – interrupts, messages, requests from users or other systems, or clock events marking the passage of time – the system, or some element of the system, must respond to them in time.
- Characterizing the events that can occur (and when they can occur) and the system or element's time-based response to those events is the essence of discussing performance.

Performance General Scenario



Portion of Scenario	Possible Values
Source	Internal or external to the system
Stimulus	Arrival of a periodic, sporadic, or stochastic event
Artifact	System or one or more components in the system.
Environment	Operational mode: normal, emergency, peak load, overload.
Response	Process events, change level of service
Response Measure	Latency, deadline, throughput, jitter, miss rate

Sample Concrete Performance Scenario



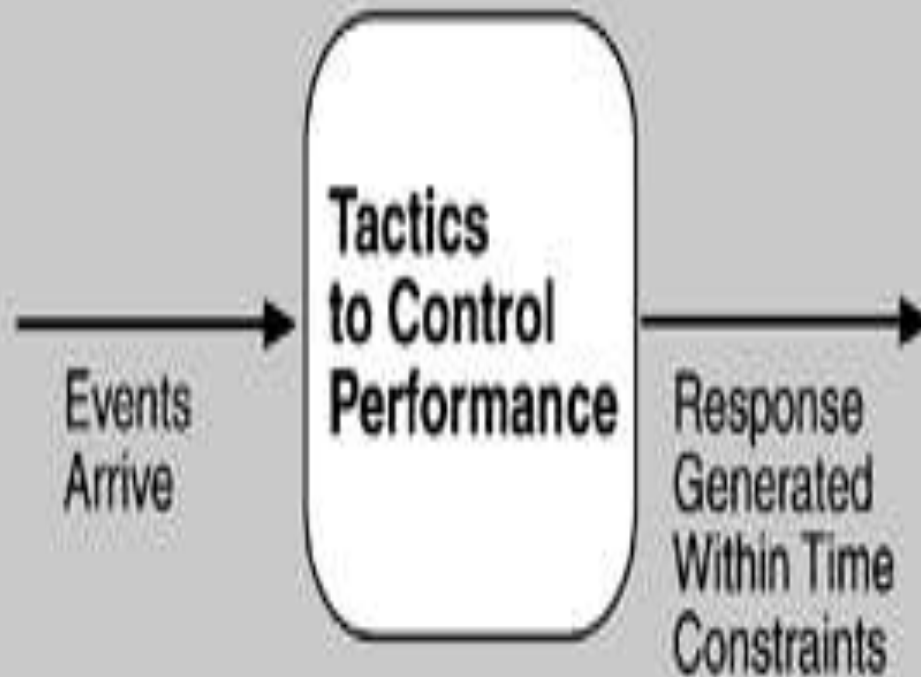
Users initiate transactions under normal operations. The system processes the transactions with an average latency of two seconds

Goal of Performance Tactics

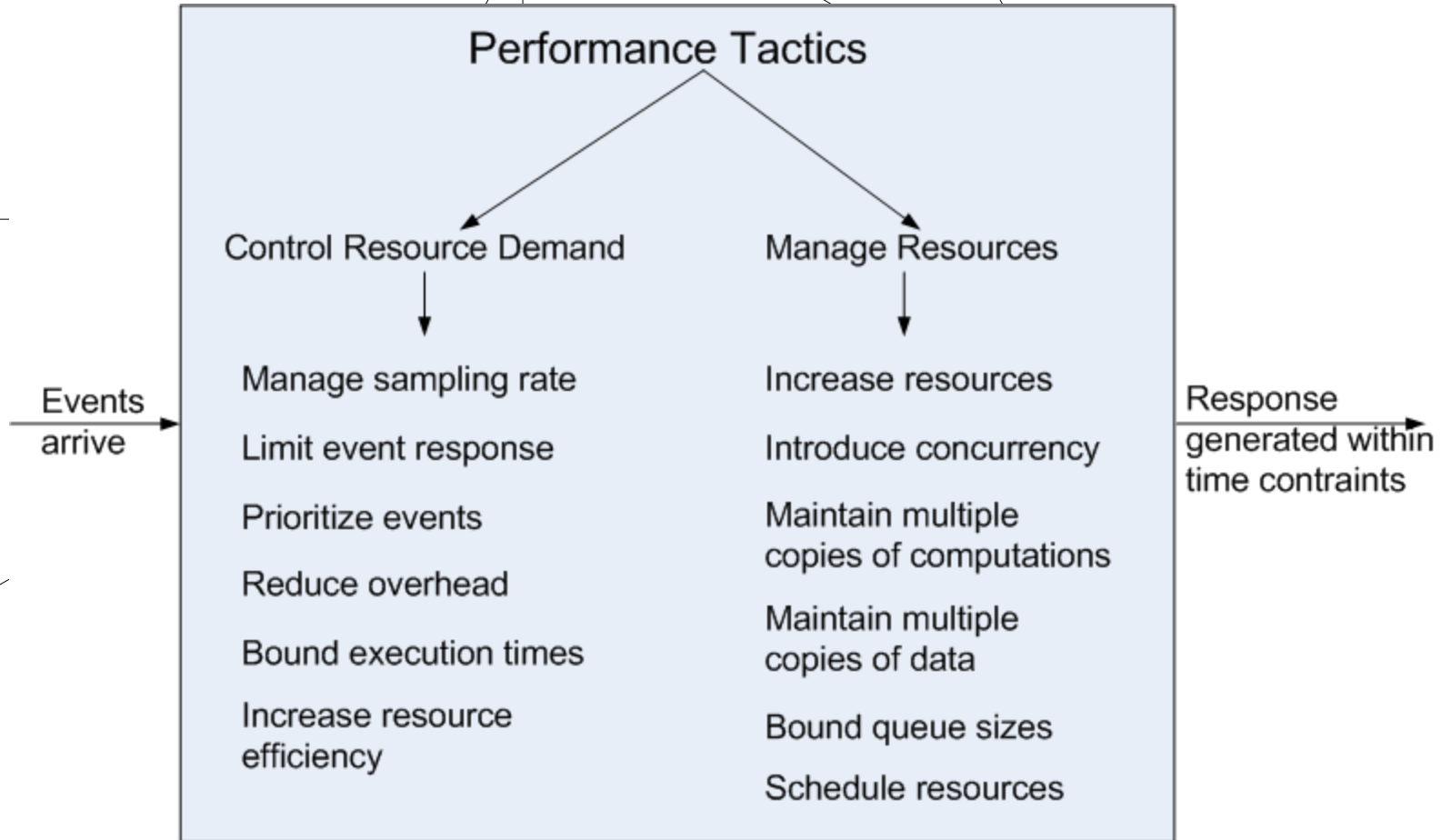


Tactics to control Performance have as their goal to generate a response to an event arriving at the system within some time-based constraint.

Goal of Performance Tactics



Performance Tactics



Control Resource Demand



- **Manage Sampling Rate:** If it is possible to reduce the sampling frequency at which a stream of data is captured, then demand can be reduced, typically with some loss of fidelity.
- **Limit Event Response:** process events only up to a set maximum rate, thereby ensuring more predictable processing when the events are actually processed.
- **Prioritize Events:** If not all events are equally important, you can impose a priority scheme that ranks events according to how important it is to service them.

Control Resource Demand



- Reduce Overhead: The use of intermediaries (important for modifiability) increases the resources consumed in processing an event stream; removing them improves latency.
- Bound Execution Times: Place a limit on how much execution time is used to respond to an event.
- Increase Resource Efficiency: Improving the algorithms used in critical areas will decrease latency.

Manage Resources



- Increase Resources: Faster processors, additional processors, additional memory, and faster networks all have the potential for reducing latency.
- Increase Concurrency: If requests can be processed in parallel, the blocked time can be reduced. Concurrency can be introduced by processing different streams of events on different threads or by creating additional threads to process different sets of activities.
- Maintain Multiple Copies of Computations: The purpose of replicas is to reduce the contention that would occur if all computations took place on a single server.

Manage Resources



- Maintain Multiple Copies of Data: keeping copies of data (possibly one a subset of the other) on storage with different access speeds.
- Bound Queue Sizes: control the maximum number of queued arrivals and consequently the resources used to process the arrivals.
- Schedule Resources: When there is contention for a resource, the resource must be scheduled.

Design Checklist for Performance



Allocation of Responsibilities

Determine the system's responsibilities that will involve heavy loading, have time-critical response requirements, are heavily used, or impact portions of the system where heavy loads or time critical events occur.

For those responsibilities, identify

- processing requirements of each responsibility and determine whether they may cause bottlenecks
- additional responsibilities to recognize and process requests appropriately including
- Responsibilities that result from a thread of control crossing process or processor boundaries.
- Responsibilities to manage the threads of control — allocation and de-allocation of threads, maintaining thread pools, and so forth.
- Responsibilities for scheduling shared resources or managing performance-related artifacts such as queues, buffers, and caches.

For the responsibilities and resources you identified, ensure that the required performance response can be met (perhaps by building a performance model to help in the evaluation).

Design Checklist for Performance



Coordination Model

Determine the elements of the system that must coordinate with each other—directly or indirectly—and choose communication and coordination mechanisms that

- supports any introduced concurrency (for example, is it thread-safe?), event prioritization, or scheduling strategy
- ensures that the required performance response can be delivered
- can capture periodic, stochastic, or sporadic event arrivals, as needed
- have the appropriate properties of the communication mechanisms, for example, stateful, stateless, synchronous, asynchronous, guaranteed delivery, throughput, or latency.

Design Checklist for Performance



Data Model

Determine those portions of the data model that will be heavily loaded, have time critical response requirements, are heavily used, or impact portions of the system where heavy loads or time critical events occur.

For those data abstractions, determine

- whether maintaining multiple copies of key data would benefit performance
- partitioning data would benefit performance
- whether reducing the processing requirements for the creation, initialization, persistence, manipulation, translation, or destruction of the enumerated data abstractions is possible
- whether adding resources to reduce bottlenecks for the creation, initialization, persistence, manipulation, translation, or destruction of the enumerated data abstractions is feasible.

innovate achieve lead

Where heavy network loading will occur, determine whether co-locating some components will reduce loading and improve overall efficiency.

Determine where introducing concurrency (that is, allocating a piece of functionality to two or more copies of a component running simultaneously) is feasible and has a significant positive effect on performance.

Determine whether the choice of threads of control and their associated responsibilities introduces bottlenecks.

Design Checklist for Performance



Resource Management

Determine which resources in your system are critical for performance. For these resources ensure they will be monitored and managed under normal and overloaded system operation.

For example

- system elements that need to be aware of, and manage, time and other performance-critical resources
- process/thread models
- prioritization of resources and access to resources
- scheduling and locking strategies
- deploying additional resources on demand to meet increased loads

Design Checklist for Performance



Binding Time

For each element that will be bound after compile time, determine the

- time necessary to complete the binding
- additional overhead introduced by using the late binding mechanism

Ensure that these values do not pose unacceptable performance penalties on the system.

Design Checklist for Performance



Choice of Technology

Will your choice of technology let you set and meet hard real time deadlines? Do you know its characteristics under load and its limits?

Does your choice of technology give you the ability to set

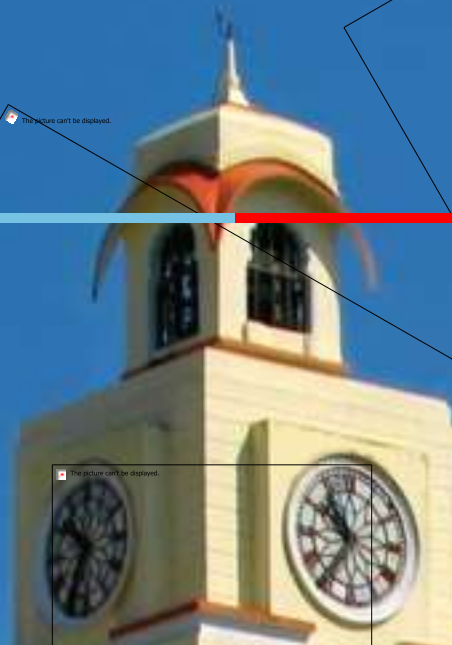
- scheduling policy
- priorities
- policies for reducing demand
- allocation of portions of the technology to processors
- other performance related parameters

Does your choice of technology introduce excessive overhead for heavily used operations?

Summary



- Performance is about the management of system resources in the face of particular types of demand to achieve acceptable timing behavior.
- Performance can be measured in terms of throughput and latency for both interactive and embedded real time systems.
- Performance can be improved by reducing demand or by managing resources more appropriately.



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Usability Module 2 – L4

Harvinder S Jabbal
CSIS, Work Integrated Learning Programs

Chapter Outline



What is Usability?

Usability General Scenario

Tactics for Usability

A Design Checklist for Usability

Summary

What is Usability?



Usability is concerned with how easy it is for the user to accomplish a desired task and the kind of user support the system provides.

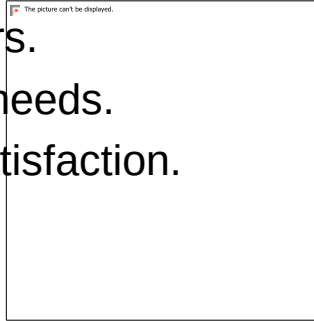
Over the years, a focus on usability has shown itself to be one of the cheapest and easiest ways to improve a system's quality (or, more precisely, the user's *perception* of quality).

What is Usability?



Usability comprises the following areas:

- Learning system features.
- Using a system efficiently.
- Minimizing the impact of errors.
- Adapting the system to user needs.
- Increasing confidence and satisfaction.



The picture can't be displayed.

Usability General Scenario

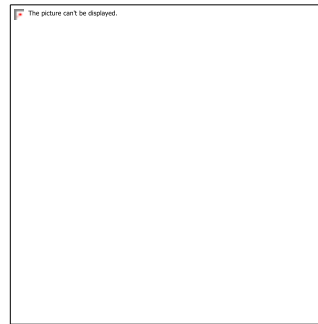


Portion of Scenario	Possible Values
Source	End user, possibly in a specialized role
Stimulus	End user tries to use a system efficiently, learn to use the system, minimize the impact of errors, adapt the system, or configure the system
Environment	Runtime or configuration time
Artifacts	System or the specific portion of the system with which the user is interacting.
Response	The system should either provide the user with the features needed or anticipate the user's needs.
Response Measure	One or more of the following: task time, number of errors, number of tasks accomplished, user satisfaction, gain of user knowledge, ratio of successful operations to total operations, or amount of time or data lost when an error occurs.

Sample Concrete Usability Scenario



- The user downloads a new application and is using it productively after two minutes of experimentation.



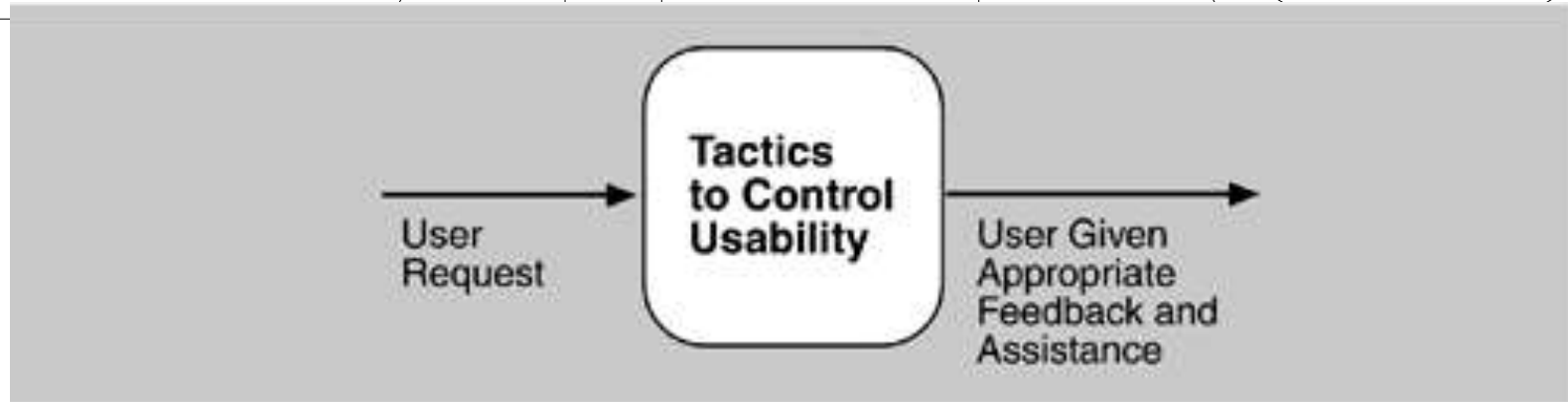
The picture can't be displayed.

Goal of Usability Tactics

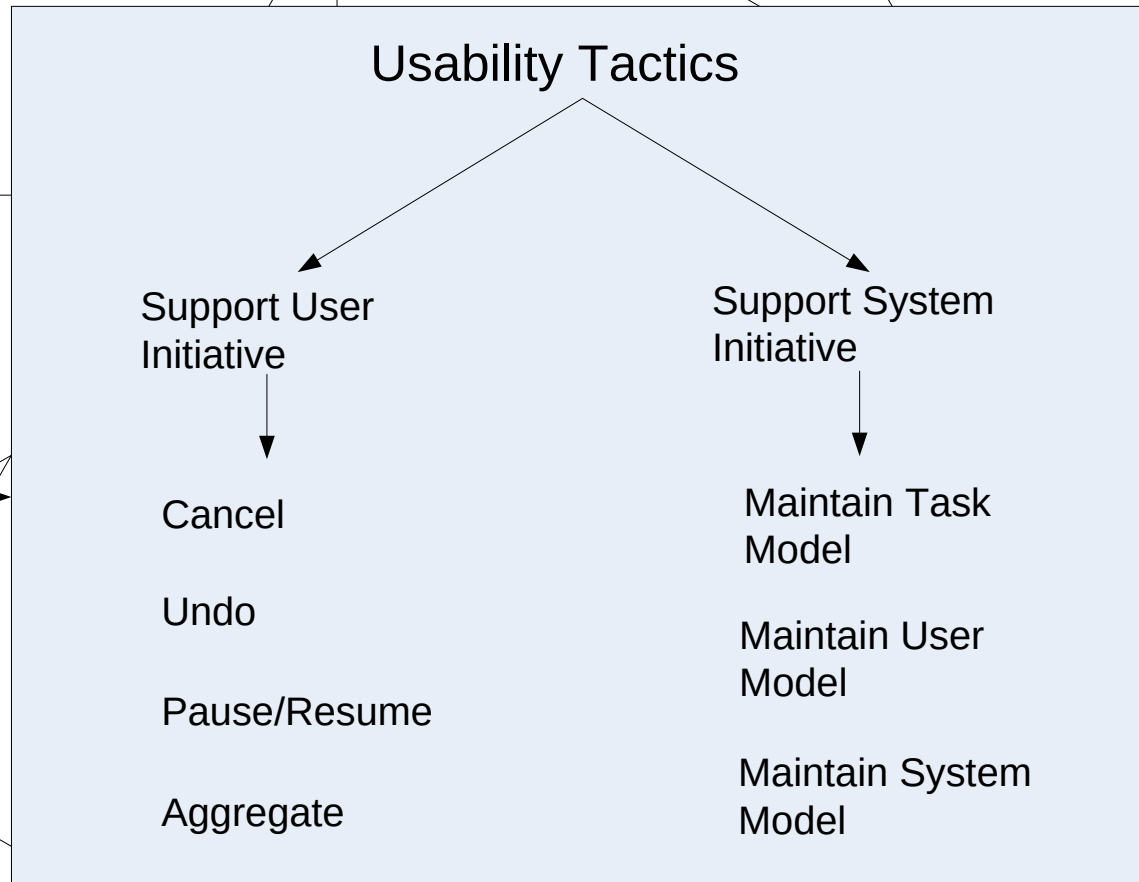


- Researchers in human-computer interaction have used the terms "user initiative," "system initiative," and "mixed initiative" to describe which of the human-computer pair takes the initiative in performing certain actions and how the interaction proceeds.
- Usability scenarios can combine initiatives from both perspectives.
- We use this distinction between user and system initiative to discuss the tactics that the architect uses to achieve the various scenarios.

Goal of Usability Tactics



Usability Tactics



User Given
Appropriate
Feedback and
Assistance

User
Request

innovate achieve lead

- 83

Support System Initiative



- Maintain Task Model: determines context so the system can have some idea of what the user is attempting and provide assistance.
- Maintain User Model: explicitly represents the user's knowledge of the system, the user's behavior in terms of expected response time, etc.
- Maintain System Model: system maintains an explicit model of itself. This is used to determine expected system behavior so that appropriate feedback can be given to the user.

Design Checklist for Usability



Allocation of Responsibilities

Ensure that additional system responsibilities have been allocated, as needed, to assist the user in

- learning how to use the system
- efficiently achieving the task at hand
- adapting and configuring the system
- recovering from user and system errors

Design Checklist for Usability



Coordination Model

Determine whether the properties of system elements' coordination—timeliness, currency, completeness, correctness, consistency—affect how a user learns to use the system, achieves goals or completes tasks, adapts and configures the system, recovers from user and system errors, increases confidence and satisfaction.

For example, can the system respond to mouse events and give semantic feedback in real time? Can long-running events be canceled in a reasonable amount of time?

Design Checklist for Usability



Data Model

Determine the major data abstractions that are involved with user-perceivable behavior.

Ensure these major data abstractions, their operations, and their properties have been designed to assist the user in achieving the task at hand, adapting and configuring the system, recovering from user and system errors, learning how to use the system, and increasing satisfaction and user confidence

For example, the data abstractions should be designed to support undo and cancel operations: the transaction granularity should not be so great that canceling or undoing an operation takes an excessively long time.

Design Checklist for Usability



Mapping Among Architectural Elements

Determine what mapping among architectural elements is visible to the end user (for example, the extent to which the end user is aware of which services are local and which are remote).

For those that are visible, determine how this affects the ways in which, or the ease with which the user will learn how to use the system, achieve the task at hand, adapt and configure the system, recover from user and system errors, and increase confidence and satisfaction.

Design Checklist for Usability



Resource Management

Determine how the user can adapt and configure the system's use of resources.

Ensure that resource limitations under all user-controlled configurations will not make users less likely to achieve their tasks. For example, attempt to avoid configurations that would result in excessively long response times.

Ensure that the level of resources will not affect the users' ability to learn how to use the system, or decrease their level of confidence and satisfaction with the system.

Design Checklist for Usability



Binding Time

Determine which binding time decisions should be under user control and ensure that users can make decisions that aid in usability.

For example, if the user can choose, at run-time, the system's configuration, or its communication protocols, or its functionality via plug-ins, you need to ensure that such choices do not adversely affect the user's ability to learn system features, use the system efficiently, minimize the impact of errors, further adapt and configure the system, or increase confidence and satisfaction.

Design Checklist for Usability



Choice of Technology

Ensure the chosen technologies help to achieve the usability scenarios that apply to your system. For example, do these technologies aid in the creation of on-line help, training materials, and user feedback collection.

How usable are any of your chosen technologies? Ensure the chosen technologies do not adversely affect the usability of the system (in terms of learning system features, using the system efficiently, minimizing the impact of errors, or adapting/configuring the system, increase confidence and satisfaction).

Summary



Architectural support for usability involves both allowing the user to take the initiative in circumstances such as cancelling a long running command, undoing a completed command, and aggregating data and commands.

To predict user or system response, the system must keep a model of the user, the system, and the task.



Other Quality Attributes

Chapter Outline



Other Important Quality Attributes

Other Categories of Quality Attributes

Software Quality Attributes and System Quality Attributes

Using Standard Lists of Quality Attributes

Dealing with “X-ability”

Summary

Other Important Quality Attributes



- **Variability:** is a special form of modifiability. It refers to the ability of a system and its supporting artifacts to support the production of a set of variants that differ from each other in a preplanned fashion.
- **Portability:** is also a special form of modifiability. Portability refers to the ease with which software that built to run on one platform can be changed to run on a different platform.
- **Development Distributability:** is the quality of designing the software to support distributed software development.

Other Important Quality Attributes



- **Scalability:** Horizontal scalability (scaling out) refers to adding more resources to logical units such as adding another server to a cluster. Vertical scalability (scaling up) refers to adding more resources to a physical unit such as adding more memory to a computer.
- **Deployability:** is concerned with how an executable arrives at a host platform and how it is invoked.
- **Mobility:** deals with the problems of movement and affordances of a platform (e.g. size, type of display, type of input devices, availability and volume of bandwidth, and battery life).

Other Important Quality Attributes



- **Monitorability:** deals with the ability of the operations staff to monitor the system while it is executing.
- **Safety:** Software safety is about the software's ability to avoid entering states that cause or lead to damage, injury, or loss of life, and to recover and limit the damage when it does enter into bad states. The architectural concerns with safety are almost identical with those for availability (i.e. preventing, detecting, and recovering from failures).

Other Categories of Quality Attributes



- **Conceptual Integrity:** refers to consistency in the design of the architecture. It contributes to the understandability of the architecture. Conceptual integrity demands that the same thing is done in the same way through the architecture.
- **Marketability:** Some systems are marketed by their architectures, and these architectures sometimes carry a meaning all their own, independent of what other quality attributes they bring to the system (e.g. service-oriented or cloud-based).

Other Categories of Quality Attributes



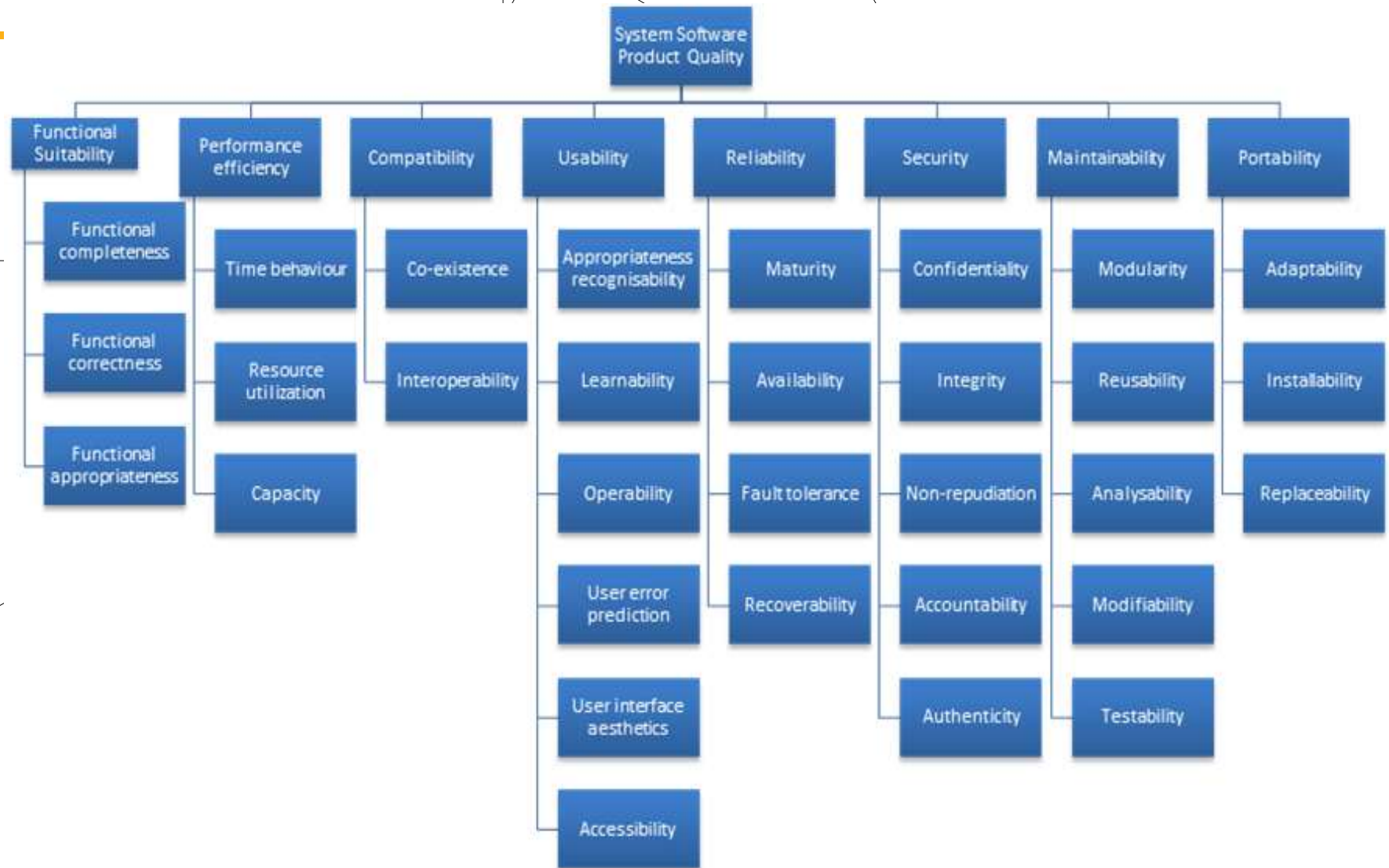
- Quality in Use: qualities that pertain to the use of the system by various stakeholders. For example
 - Effectiveness: a measure whether the system is correct
 - Efficiency: the effort and time required to develop a system
 - Freedom from risk: degree to which a product or system affects economic status, human life, health, or the environment

Software Quality Attributes and System Quality Attributes



- Physical systems, such as aircraft or automobiles or kitchen appliances, that rely on software embedded within are designed to meet a whole other litany of quality attributes: weight, size, electric consumption, power output, pollution output, weather resistance, battery life, and on and on.
- The software architecture can have a substantial effect on the system's quality attributes.

Standard Lists of Quality Attributes

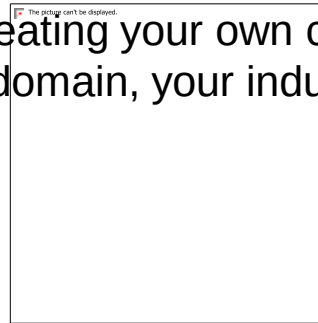


Standard Lists of Quality Attributes



Advantages:

- Can be helpful checklists to assist requirements gatherers in making sure that no important needs were overlooked.
- Can serve as the basis for creating your own checklist that contains the quality attributes of concern in your domain, your industry, your organization, your products, ...



The picture can't be displayed.

Standard Lists of Quality Attributes



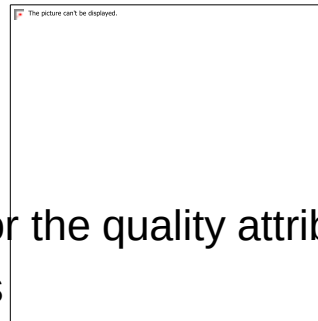
Disadvantages:

- No list will ever be complete.
- Lists often generate more controversy than understanding.
- Lists often purport to be *taxonomies*. But what is a denial-of-service attack?
- They force architects to pay attention to every quality attribute on the list, even if only to finally decide that the particular quality attribute is irrelevant to their system.

Dealing with “X-ability”



- Suppose you must deal with a quality attribute for which there is no compact body of knowledge, e.g. green computing.
- What do you do?
 1. Model the quality attribute
 2. Assemble a set of tactics for the quality attribute
 3. Construct design checklists



The picture can't be displayed.

Summary



- There are many other quality attributes than the seven that we cover in detail.
- Taxonomies of attributes may offer some help, but their disadvantages often outweigh their advantages.
- You may need to design or analyze a system for a “new” quality attribute. While this may be challenging, it is doable.

innovate

achieve

lead



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Security Module 2 – L5

- Harvinder S Jabbal
- CSIS, Work Integrated Learning Programs

August 1, 2026

SEZG651/SSZG653 Software
Architectures

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Chapter Outline



What is Security?

Security General Scenario

Tactics for Security

A Design Checklist for Security

Summary

What is Security?



- Security is a measure of the system's ability to protect data and information from unauthorized access while still providing access to people and systems that are authorized.
- An action taken against a computer system with the intention of doing harm is called an *attack* and can take a number of forms.
- It may be an unauthorized attempt to access data or services or to modify data, or it may be intended to deny services to legitimate users.

What is Security?



Security has three main characteristics, called CIA:

- Confidentiality is the property that data or services are protected from unauthorized access. For example, a hacker cannot access your income tax returns on a government computer.
- Integrity is the property that data or services are not subject to unauthorized manipulation. For example, your grade has not been changed since your instructor assigned it.
- Availability is the property that the system will be available for legitimate use. For example, a denial-of-service attack won't prevent you from ordering a book from an online bookstore.

Other characteristics that support CIA are

- Authentication verifies the identities of the parties to a transaction and checks if they are truly who they claim to be. For example, when you get an e-mail purporting to come from a bank, authentication guarantees that it actually comes from the bank.
- Nonrepudiation guarantees that the sender of a message cannot later deny having sent the message and that the recipient cannot deny having received the message. For example, you cannot deny ordering something from the Internet, or the merchant cannot disclaim getting your order.
- Authorization grants a user the privileges to perform a task. For example, an online banking system authorizes a legitimate user to access his account.

Security General Scenario



Portion of Scenario	Possible Values
Source	Human or another system which may have been previously identified (either correctly or incorrectly) or may be currently unknown. A human attacker may be from outside the organization or from inside the organization.
Stimulus	Unauthorized attempt is made to display data, change or delete data, access system services, change the system's behavior, or reduce availability.
Artifact	System services; data within the system; a component or resources of the system; data produced or consumed by the system
Environment	The system is either online or offline, connected to or disconnected from a network, behind a firewall or open to a network, fully operational, partially operational, or not operational
Response	Transactions are carried out in a fashion such that • data or services are protected from unauthorized access; • data or services are not being manipulated without authorization; • parties to a transaction are identified with assurance; • the parties to the transaction cannot repudiate their involvements; • the data, resources, and system services will be available for legitimate use. The system tracks activities within it by • recording access or modification, • recording attempts to access data, resources or services, • notifying appropriate entities (people or systems) when an apparent attack is occurring.
Response Measure	One or more of the following • how much of a system is compromised when a particular component or data value is compromised, • how much time passed before an attack was detected, • how many attacks were resisted, • how long does it take to recover from a successful attack, • how much data is vulnerable to a particular attack

Sample Concrete Security Scenario



- A disgruntled employee from a remote location attempts to modify the pay rate table during normal operations. The system maintains an audit trail and the correct data is restored within a day.

Goal of Security Tactics



- One method for thinking about system security is to think about physical security.
- Secure installations have limited access to them (e.g., by using security checkpoints), have means of detecting intruders (e.g., by requiring legitimate visitors to wear badges), have deterrence mechanisms such as armed guards, have reaction mechanisms such as automatic locking of doors and have recovery mechanisms such as off-site back up.
- This leads to our four categories of tactics: detect, resist, react, and recover.

Goal of Security Tactics



Detect Attacks



- Detect Intrusion: compare network traffic or service request patterns *within* a system to a set of signatures or known patterns of malicious behavior stored in a database.
- Detect Service Denial: comparison of the pattern or signature of network traffic *coming into* a system to historic profiles of known Denial of Service (DoS) attacks.
- Verify Message Integrity: use techniques such as checksums or hash values to verify the integrity of messages, resource files, deployment files, and configuration files.
- Detect Message Delay: checking the time that it takes to deliver a message, it is possible to detect suspicious timing behavior.

Resist Attacks



- **Identify Actors:** identify the source of any external input to the system.
- **Authenticate Actors:** ensure that an actor (user or a remote computer) is actually who or what it purports to be.
- **Authorize Actors:** ensuring that an authenticated actor has the rights to access and modify either data or services.
- **Limit Access:** limiting access to resources such as memory, network connections, or access points.

Resist Attacks



- Limit Exposure: minimize the attack surface of a system by having the fewest possible number of access points.
- Encrypt Data: apply some form of encryption to data and to communication.
- Separate Entities: can be done through physical separation on different servers attached to different networks, the use of virtual machines, or an “air gap”.
- Change Default Settings: Force the user to change settings assigned by default.

React to Attacks



- Revoke Access: limit access to sensitive resources, even for normally legitimate users and uses, if an attack is suspected.
- Lock Computer: limit access to a resource if there are repeated failed attempts to access it.
- Inform Actors: notify operators, other personnel, or cooperating systems when an attack is suspected or detected.

Recover From Attacks



- In addition to the Availability tactics for recovery of failed resources there is Audit.
- Audit: keep a record of user and system actions and their effects, to help trace the actions of, and to identify, an attacker.

Design Checklist for Security



Allocation of Responsibilities

Determine which system responsibilities need to be secure. For each of these responsibilities ensure that additional responsibilities have been allocated to:

- identify the actor
- authenticate the actor
- authorize actors
- grant or deny access to data or services
- record attempts to access or modify data or services
- encrypt data
- recognize reduced availability for resources or services and inform appropriate personnel and restrict access
- recover from an attack
- verify checksums and hash values

Design Checklist for Security



Coordination Model

Determine mechanisms required to communicate and coordinate with other systems or individuals. For these communications, ensure that mechanisms for authenticating and authorizing the actor or system, and encrypting data for transmission across the connection are in place.

Ensure also that mechanisms exist for monitoring and recognizing unexpectedly high demands for resources or services as well as mechanisms for restricting or terminating the connection.

Design Checklist for Security



Data Model

Determine the sensitivity of different data fields. For each data abstraction

- Ensure that data of different sensitivity is separated.
- Ensure that data of different sensitivity has different access rights and that access rights are checked prior to access.
- Ensure that access to sensitive data is logged and that the log file is suitably protected.
- Ensure that data is suitably encrypted and that keys are separated from the encrypted data.
- Ensure that data can be restored if it is inappropriately modified.

Design Checklist for Security



Mapping Among Architectural Elements

Determine how alternative mappings of architectural elements may change how an individual or system may read, write, or modify data, access system services or resources, or reduce their availability. Determine how alternative mappings may affect the recording of access to data, services or resources and the recognition of high demands for resources.

For each such mapping, ensure that there are responsibilities to

- identify an actor
- authenticate an actor
- authorize actors
- grant or deny access to data or services
- record attempts to access or modify data or services
- encrypt data
- recognize reduced availability for resources or services, inform appropriate personnel, and restrict access
- recover from an attack

Design Checklist for Security



Resource Management

Determine the system resources required to identify and monitor a system or an individual who is internal or external, authorized or not authorized, with access to specific resources or all resources.

Determine the resources required to authenticate the actor, grant or deny access to data or resources, notify appropriate entities, record attempts to access data or resources, encrypt data, recognize high demand for resources, inform users or systems, and restrict access.

For these resources consider whether an external entity can access or exhaust a critical resource; how to monitor the resource; how to manage resource utilization; how to log resource utilization and ensure that there are sufficient resources to perform necessary security operations.

Ensure that a contaminated element can be prevented from contaminating other elements.

Ensure that shared resources are not used for passing sensitive data from an actor with access rights to that data to an actor without access rights.

Design Checklist for Security



Binding Time

Determine cases where an instance of a late bound component may be untrusted.

For such cases ensure that late bound components can be qualified, that is, if ownership certificates for late bound components are required, there are appropriate mechanisms to manage and validate them; that access to late bound data and services can be managed; that access by late bound components to data and services can be blocked; that mechanisms to record the access, modification, and attempts to access data or services by late bound components are in place; and that system data is encrypted where the keys are intentionally withheld for late bound components

Design Checklist for Security



Choice of Technology

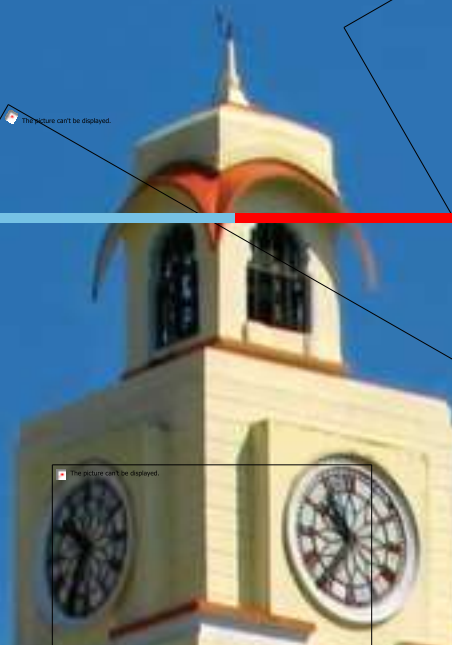
Determine what technologies are available to help user authentication, data access rights, resource protection, data encryption.

Ensure that your chosen technologies support the tactics relevant for your security needs.

Summary



- Attacks against a system can be characterized as attacks against the confidentiality, integrity, or availability of a system or its data.
- This leads to many of the tactics used to achieve security. Identifying, authenticating, and authorizing actors are tactics intended to determine which users or systems are entitled to what kind of access to a system.
- No security tactic is foolproof and systems *will* be compromised. Hence, tactics exist to detect an attack, limit the spread of any attack, and to react and recover from an attack.



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Modifiability Module 2 – L6

Harvinder S Jabbal
CSIS, Work Integrated Learning Programs

Chapter Outline



What is Modifiability?

Modifiability General Scenario

Tactics for Modifiability

A Design Checklist for Modifiability

Summary

What is Modifiability?



Modifiability is about change and our interest in it is in the cost and risk of making changes.

To plan for modifiability, an architect has to consider three questions:

- What can change?
- What is the likelihood of the change?
- When is the change made and who makes it?

Modifiability General Scenario



Portion of Scenario	Possible Values
Source	End user, developer, system administrator
Stimulus	A directive to add/delete/modify functionality, or change a quality attribute, capacity, or technology
Artifacts	Code, data, interfaces, components, resources, configurations, ...
Environment	Runtime, compile time, build time, initiation time, design time
Response	One or more of the following: • make modification • test modification • deploy modification
Response Measure	Cost in terms of: • number, size, complexity of affected artifacts • effort • calendar time • money (direct outlay or opportunity cost) • extent to which this modification affects other functions or quality attributes • new defects introduced

Auguts 1, 2026

Sample Concrete Modifiability Scenario



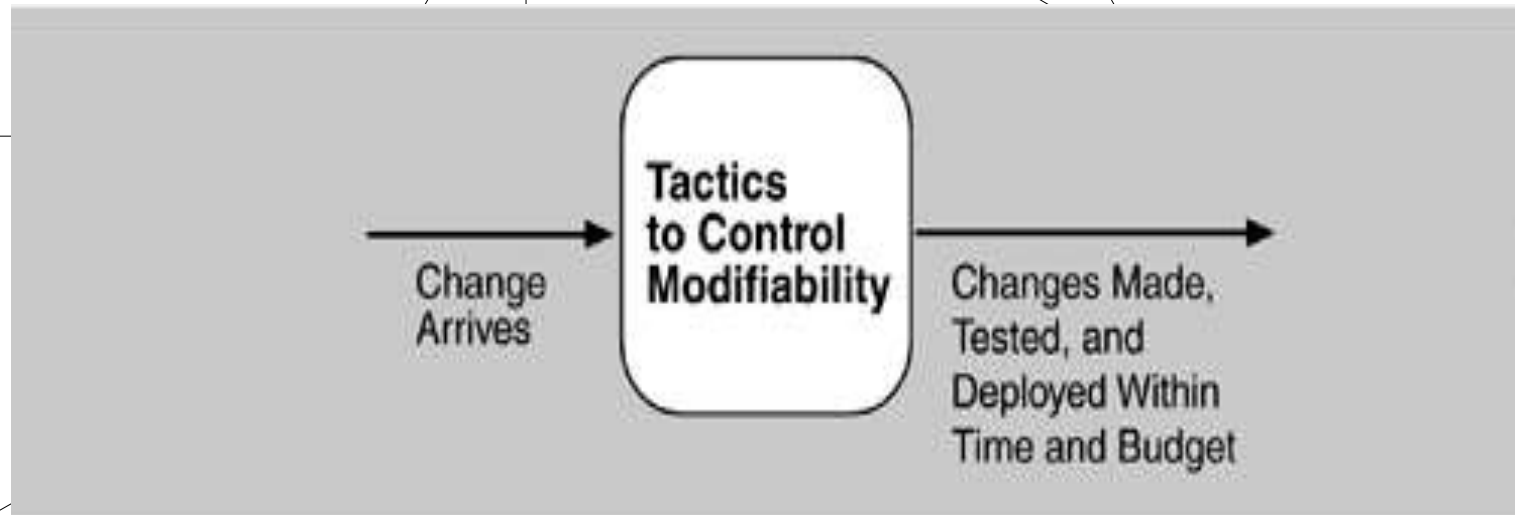
The developer wishes to change the user interface by modifying the code at design time. The modifications are made with no side effects within three hours.

Goal of Modifiability Tactics



Tactics to control modifiability have as their goal controlling the complexity of making changes, as well as the time and cost to make changes.

Goal of Modifiability Tactics



Reduce Size of a Module



Split Module: If the module being modified includes a great deal of capability, the modification costs will likely be high. Refining the module into several smaller modules should reduce the average cost of future changes.

Increase Cohesion



Increase Semantic Coherence: If the responsibilities A and B in a module do not serve the same purpose, they should be placed in different modules. This may involve creating a new module or it may involve moving a responsibility to an existing module.

Reduce Coupling



Encapsulate: Encapsulation introduces an explicit interface to a module. This interface includes an API and its associated responsibilities, such as “perform a syntactic transformation on an input parameter to an internal representation.”

Use an Intermediary: Given a dependency between responsibility A and responsibility B (for example, carrying out A first requires carrying out B), the dependency can be broken by using an intermediary.

Reduce Coupling



Restrict Dependencies: restricts the modules which a given module interacts with or depends on.

Refactor: undertaken when two modules are affected by the same change because they are (at least partial) duplicates of each other.

Abstract Common Services: where two modules provide not-quite-the-same but similar services, it may be cost-effective to implement the services just once in a more general (abstract) form.

Defer Binding



In general, the later in the life cycle we can bind values, the better.

If we design artifacts with built-in flexibility, then exercising that flexibility is usually cheaper than hand-coding a specific change.

However, putting the mechanisms in place to facilitate that late binding tends to be more expensive.

Design Checklist for Modifiability



Allocation of Responsibilities

Determine which changes or categories of changes are likely to occur through consideration of changes in technical, legal, social, business, and customer forces. For each potential change or category of changes

- Determine the responsibilities that would need to be added, modified, or deleted to make the change.
- Determine what other responsibilities are impacted by the change.
- Determine an allocation of responsibilities to modules that places, as much as possible, responsibilities that will be changed (or impacted by the change) together in the same module, and places responsibilities that will be changed at different times in separate modules.

Design Checklist for Modifiability



Coordination Model

Determine which functionality or quality attribute can change at runtime and how this affects coordination; for example, will the information being communicated change at run-time, or will the communication protocol change at run-time? If so, ensure that such changes affect a small number set of modules.

Determine which devices, protocols, and communication paths used for coordination are likely to change. For those devices, protocols, and communication paths, ensure that the impact of changes will be limited to a small set of modules.

For those elements for which modifiability is a concern, use a coordination model that reduces coupling such as publish/subscribe, defers bindings such as enterprise service bus, or restricts dependencies such as broadcast.

Design Checklist for Modifiability



Data Model

Determine which changes (or categories of changes) to the data abstractions, their operations, or their properties are likely to occur. Also determine which changes or categories of changes to these data abstractions will involve their creation, initialization, persistence, manipulation, translation, or destruction.

For each change or category of change, determine if the changes will be made by an end user, system administrator, or developer. For those changes made by an end user or administrator, ensure that the necessary attributes are visible to that user and that the user has the correct privileges to modify the data, its operations, or its properties.

For each potential change or category of change

- determine which data abstractions need to be added, modified, or deleted
- determine whether there would be any changes to the creation, initialization, persistence, manipulation, translation, or destruction of these data abstractions
- determine which other data abstractions are impacted by the change. For these additional abstractions, determine whether the impact would be on their operations, properties, creation, initialization, persistence, manipulation, translation, or destruction.
- ensure an allocation of data abstractions that minimizes the number and severity of modifications to the abstractions by the potential changes

Design your data model so that items allocated to each element of the data model are likely to change together.

Design Checklist for Modifiability



Mapping Among Architectural Elements

Determine if it is desirable to change the way in which functionality is mapped to computational elements (e.g. processes, threads, processors) at runtime, compile time, design time, or build time.

Determine the extent of modifications necessary to accommodate the addition, deletion, or modification of a function or a quality attribute. This might involve a determination of, for example:

- execution dependencies
- assignment of data to databases
- assignment of runtime elements to processes, threads, or processors

Ensure that such changes are performed with mechanisms that utilize deferred binding of mapping decisions.

Design Checklist for Modifiability



Resource Management

Determine how the addition, deletion, or modification of a responsibility or quality attribute will affect resource usage. This involves, for example,

- determining what changes might introduce new resources or remove old ones or affect existing resource usage.
- determining what resource limits will change and how

Ensure that the resources after the modification are sufficient to meet the system requirements.

Encapsulate all resource managers and ensure that the policies implemented by those resource managers utilize are themselves encapsulated and bindings are deferred to the extent possible.

Design Checklist for Modifiability



Binding Time

For each change or category of change

- Determine the latest time at which the change will need to be made.
- Choose a defer-binding mechanism (see Section 7.2.4) that delivers the appropriate capability at the time chosen.
- Determine the cost of introducing the mechanism and the cost of making changes using the chosen mechanism
- Do not introduce so many binding choices that change is impeded because the dependencies among the choices are complex and unknown.

Design Checklist for Modifiability



Choice of Technology

Determine what modifications are made easier or harder by your technology choices.

- Will your technology choices help to make, test, and deploy modifications?
- How easy is it to modify your choice of technologies itself (in case some of these technologies change or become obsolete)?

Choose your technologies to support the most likely modifications. For example, an Enterprise Service Bus makes it easier to change how elements are connected but may introduce vendor lock in.

Summary



Modifiability deals with change and the cost in time or money of making a change, including the extent to which this modification affects other functions or quality attributes.

Tactics to reduce the cost of making a change include making modules smaller, increasing cohesion, and reducing coupling. Deferring binding will also reduce the cost of making a change.